

APIs with Lambda + API Gateway

Melvin green

Create method

Method details

Method type
GET

Integration type

☒ **Lambda function**
Integrate your API with a Lambda function.


☐ **HTTP**
Integrate with an existing HTTP endpoint.


☐ **AWS service**
Integrate with an AWS Service.


☐ **VPC link**
Integrate with a resource that isn't accessible over the public internet.


☒ **Lambda proxy integration**
Send the request to your Lambda function as a structured event.

Response transfer mode [Info](#)

☒ **Buffered**
Wait to receive the complete response before beginning transmission.

☐ **Stream**
Send portions of the response without waiting for the complete response.

Lambda function
Provide the Lambda function name or alias. You can also provide an ARN from another account.

us-east-1 

 `arn:aws:lambda:us-east-1:157975550086:function:RetrieveUserData` 

Introducing Today's Project!

In this project, I demonstrate the logic tier of a three-tier application architecture. The goal of this project is to learn how to build and deploy serverless backend logic using AWS Lambda and API Gateway, and to understand how these services fit into a broader three-tier design (presentation, logic, and data tiers).

Tools and concepts

Here's the recruiter-facing version as a single paragraph: Through this project, I gained hands-on experience with serverless architecture and RESTful API design using AWS Lambda and Amazon API Gateway. I learned how to write and deploy Lambda functions to handle backend logic without managing servers, how to configure API Gateway to route HTTP requests to the appropriate Lambda function, and how to structure an API using resources and methods (GET, POST, PUT, DELETE) that follow REST conventions. I also learned how API deployment stages work, including deploying to a live (Prod) environment accessible via a public invoke URL. This project gave me practical experience with the logic tier of a three-tier architecture and a foundational understanding of how modern, scalable backend systems are built on AWS.

Project reflection

This project took me approximately an hour. The most challenging part was creating the Lambda function because I still need to learn more scripting. It was most rewarding to be able to create my first API after learning about it in my CompTIA Network+ course.

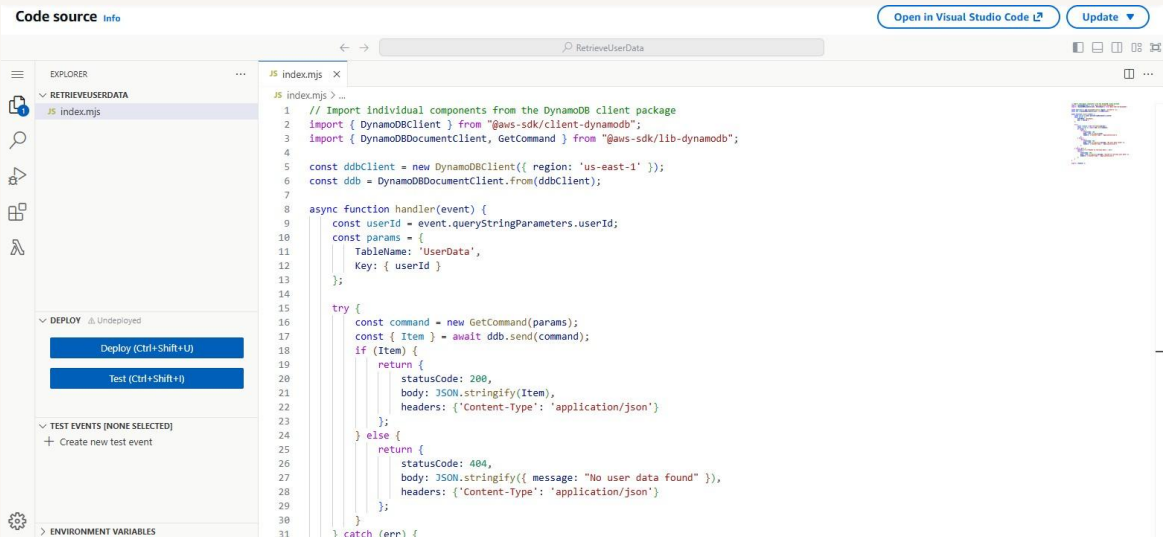
I chose to do this project today because these are key skills in becoming a cloud support engineer.

Lambda functions

AWS Lambda is a serverless compute service that lets you run code without provisioning or managing any servers. In this project, I'm using Lambda to fetch data from a database and return it to users, allowing the backend to scale automatically and only run when needed.

The code I added to this Lambda function retrieves specific user data based on the 'userId' parameter and returns that data as a response. This demonstrates a common backend

pattern used in real-world applications: accepting an identifier, querying a database for the matching record, and returning structured data to the client.



The screenshot shows the AWS Lambda console interface for a function named 'RetrieveUserData'. The 'Code source' tab is selected, displaying the JavaScript code for 'index.mjs'. The code imports the AWS SDK for DynamoDB, initializes a client for the 'us-east-1' region, and defines an asynchronous handler function. This function extracts the 'userId' from the query string parameters and uses the DynamoDB client to retrieve a record from the 'UserData' table. If a record is found, it returns a 200 status code with the record's JSON data. If not found, it returns a 404 status code with a message 'No user data found'. The code also includes error handling with a catch block.

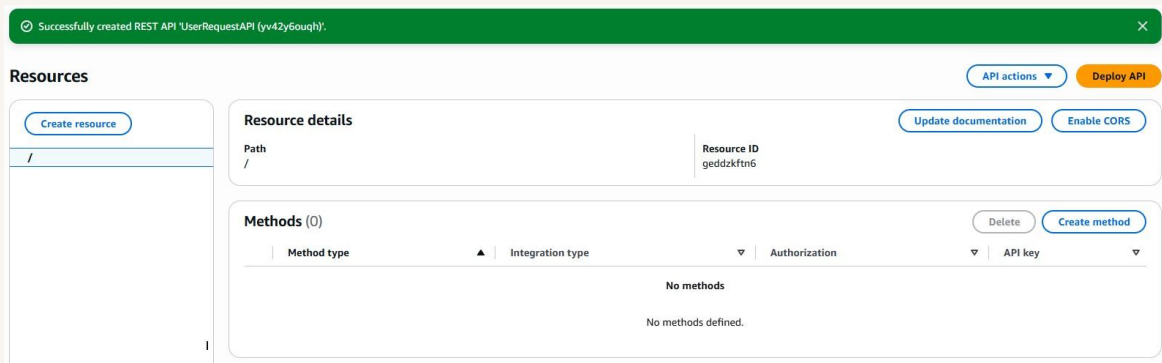
```
1 // Import individual components from the DynamoDB client package
2 import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
3 import { DynamoDBDocumentClient, GetCommand } from "@aws-sdk/lib-dynamodb";
4
5 const ddbClient = new DynamoDBClient({ region: 'us-east-1' });
6 const ddb = DynamoDBDocumentClient.from(ddbClient);
7
8 async function handler(event) {
9   const userId = event.queryStringParameters.userId;
10   const params = {
11     TableName: 'UserData',
12     Key: { userId }
13   };
14
15   try {
16     const command = new GetCommand(params);
17     const { Item } = await ddb.send(command);
18     if (Item) {
19       return {
20         statusCode: 200,
21         body: JSON.stringify(Item),
22         headers: { 'Content-Type': 'application/json' }
23       };
24     } else {
25       return {
26         statusCode: 404,
27         body: JSON.stringify({ message: "No user data found" }),
28         headers: { 'Content-Type': 'application/json' }
29       };
30     }
31   } catch (err) {
```

API Gateway

APIs are software systems that allow other systems to talk to each other. Think of an API like a messenger that carries requests and responses between systems. There are different types of APIs, such as REST APIs, HTTP APIs, and WebSocket APIs. The API I will be using today is a REST API.

Amazon API Gateway is an AWS service that makes it easy for developers to create, publish, maintain, monitor, and secure APIs at any scale. I'm using API Gateway in this project to carry requests from the user's browser to my Lambda function.

When a user makes a request, it is received by the API Gateway and then forwarded to the Lambda function for processing. After the request is processed by Lambda, the response is sent through API Gateway back to the user.



API Resources and Methods

An API is made up of resources, which are individual endpoints within my API that handle different parts of its functionality.

Each resource consists of methods, which are actions you can perform on a resource, based on standard HTTP methods such as GET, POST, PUT, and DELETE.

I created a GET method to retrieve the data.

Create method

Method details

Method type

GET

Integration type

☒

Lambda function

Integrate your API with a Lambda function.



☐

HTTP

Integrate with an existing HTTP endpoint.



☐

AWS service

Integrate with an AWS Service.



☐

VPC link

Integrate with a resource that isn't accessible over the public internet.



☒

Lambda proxy integration

Send the request to your Lambda function as a structured event.

Response transfer mode [Info](#)

☒

Buffered

Wait to receive the complete response before beginning transmission.

☐

Stream

Send portions of the response without waiting for the complete response.

Lambda function

Provide the Lambda function name or alias. You can also provide an ARN from another account.

us-east-1

arn:aws:lambda:us-east-1:157975550086:function:RetrieveUserData

API Deployment

When you deploy an API, you deploy it to a specific stage. A stage is a snapshot of my API at a specific point in time. I deployed to Prod, which is the live environment where my API is fully functional and handling live traffic.

To visit my API, I used the invoke URL. The API displayed an error because I haven't deployed my DynamoDB table yet, which I will be working on in the next project.

Deploy API

Create or select a stage where your API will be deployed. You can use the deployment history to revert or change the active deployment for a stage. [Learn more](#)

Stage

New stage

Stage name

prod

Deployment description

A new stage will be created with the default settings. Edit your stage settings on the **Stage** page.

[Cancel](#) [Deploy](#)

API Documentation

For my project's extension, I am writing API documentation because it is crucial for developers to understand how to use the API correctly and efficiently. You can do this in the Documentation tab under the API: UserRequestAPI section in API Gateway.

Once I have prepared my documentation, I can publish it to a special file type like Swagger or OpenAPI. You have to publish your API documentation to a specific stage because selecting the stage when publishing ensures that your documentation is consistent with the API version deployed to that stage.

My published and downloaded documentation showed the high-level overview of the changes made in the API I just generated.

```
    "title" : "Empty Schema"
  },
  "x-amazon-apigateway-documentation" : {
    "version" : "1",
    "createdDate" : "2026-07-17T02:07:50Z",
    "documentationParts" : [ {
      "location" : {
        "type" : "API"
      },
      "properties" : {
        "description" : "The UserRequestAPI manages user data retrieval and manipulation. It supports operations to retrieve user details based on unique identifiers."
        "baseUrl" : "https://yv42y0ouqh.execute-api.us-east-1.amazonaws.com/prod"
      }
    }
  ]
}
```